

Part 1 - Introduction & Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to describe the system architecture, technical decisions, and implementation plan for the **Sales Lead Management Tool**, which is developed as part of the Keyloop Technical Assessment.

This document explains how the system is designed, why specific technologies are selected, and how the backend solution satisfies the functional and non-functional requirements of scenario C.

1.2 Project Overview

Modern dealerships receive a large number of customer inquiries through their websites every day.

Sales representatives need an efficient system to:

- view all incoming leads,
- track customer interactions,
- record follow-up activities,
- monitor lead status.

Without a centralized management system, sales activities are often recorded manually, making it difficult to monitor customer engagement and sales performance.

The proposed Sales Lead Management Tool provides a lightweight web application that centralizes all lead information and customer follow-up activities.

1.3 Objectives

The application aims to:

- Display all incoming sales leads.
- Display complete lead information.
- Record follow-up activities.
- Maintain activity history.
- Provide a clean and maintainable architecture.

- Demonstrate modern software engineering practices.

2. Problem Statement

Many dealerships still manage sales leads using spreadsheets, emails, or handwritten notes.

This manual workflow introduces several problems:

- Customer information is scattered.
- Follow-up history is difficult to track.
- Salespersons cannot easily determine the current lead status.
- Managers have limited visibility into customer interactions.

The objective of this project is to develop a centralized web application that enables salespersons to efficiently manage incoming leads and their associated activities.

3. Scope

The project focuses on one dealership and this repository contains the backend service layer for the assessment.

Only sales lead management is included.

In Scope

- Lead List
- Lead Detail
- Activity Timeline
- Add Activity
- Search Leads
- Filter Leads
- Persistent Database

Out of Scope

- Authentication
- Authorization
- Email Notification
- SMS Notification
- Multi-dealership support

- Reporting Dashboard

4. Functional Requirements

ID	Requirement
FR-01	Display all incoming sales leads
FR-02	View complete lead information
FR-03	Display chronological activity timeline
FR-04	Add a new follow-up activity
FR-05	Store all activities permanently
FR-06	Search leads by name
FR-07	Filter leads by status

5. Non-Functional Requirements

Category	Requirement
Performance	Response time < 500ms under normal load
Availability	REST API available during application runtime
Reliability	Data stored in PostgreSQL
Scalability	Layered architecture supports future expansion
Maintainability	Clean Architecture principles
Security	Server-side validation
Observability	Logging and exception handling
Testability	Business logic covered by unit tests

6. Assumptions

Since the assessment intentionally leaves some requirements open to interpretation, the following assumptions are made.

A1	Each Lead belongs to exactly one Salesperson.
A2	A Lead may have multiple follow-up activities.
A3	Activities cannot be edited after creation. Only new activities can be added
A4	Only one dealership is supported.
A5	Each activity records its creation timestamp automatically.

7. User Roles

Only one role exists in this assessment.

Salesperson

Responsibilities:

- View lead list
- Search leads
- View lead details
- Add activities
- Monitor activity history

8. User Stories

US-01	As a Salesperson, I want to view all incoming leads so that I know which customers require follow-up.
US-02	As a Salesperson, I want to view lead details so that I can understand the customer's information.
US-03	As a Salesperson, I want to record a follow-up activity so that customer interactions are stored permanently.
US-04	As a Salesperson, I want to search leads so that I can quickly find customers.
US-05	As a Salesperson, I want to filter leads by status so that I can prioritize my work.

9. Success Criteria

The application is considered successful when:

- All incoming leads can be displayed.
- Lead details are available.
- Activities are stored permanently.
- Activity timeline updates immediately after adding a new activity.
- REST APIs function correctly.
- Unit tests pass successfully.
- Documentation is complete.

PART 2 - System Design

2.1 Domain Model

2.1.1 Domain Overview

The Sales Lead Management Tool consists of three core business entities:

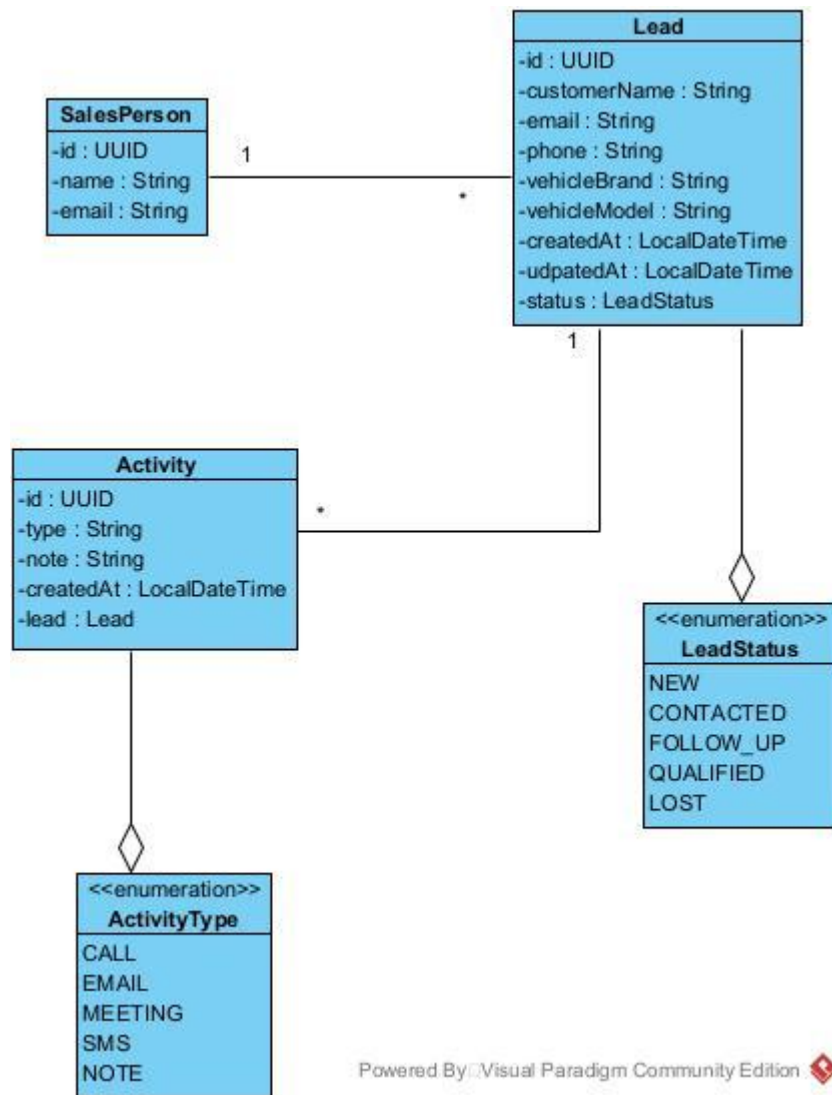
- SalesPerson
- Lead
- Activity

Each Lead is assigned to one SalesPerson.

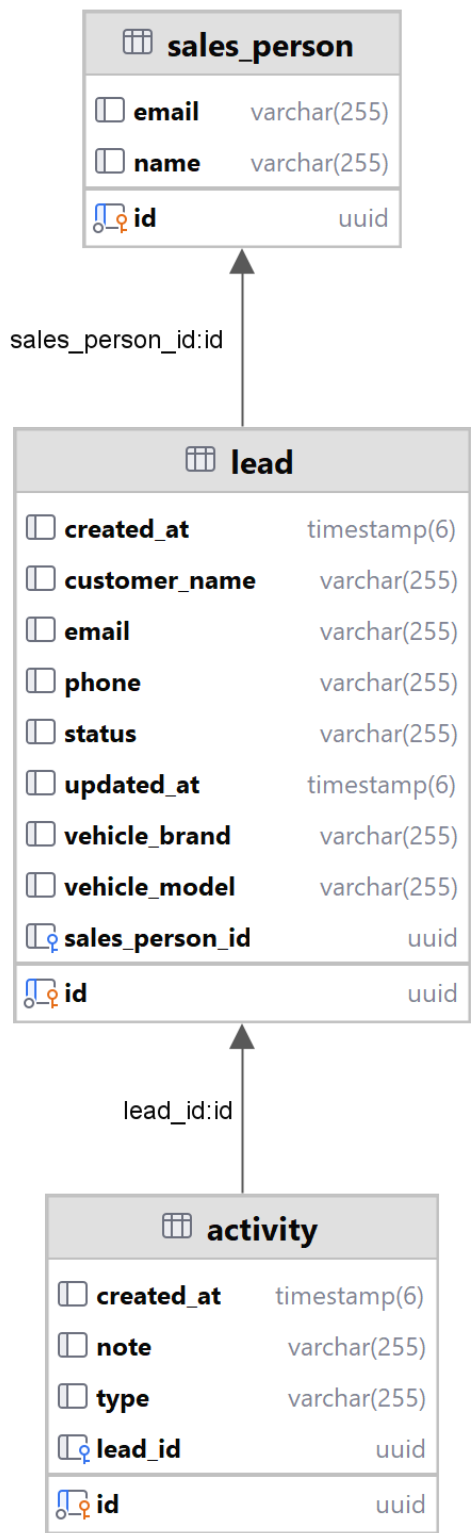
Each Lead may contain multiple follow-up Activities.

Activities represent chronological interactions between the salesperson and the customer.

2.1.2 Domain Model Diagram



2.1.3 Database Design



PART 3 - System Architecture

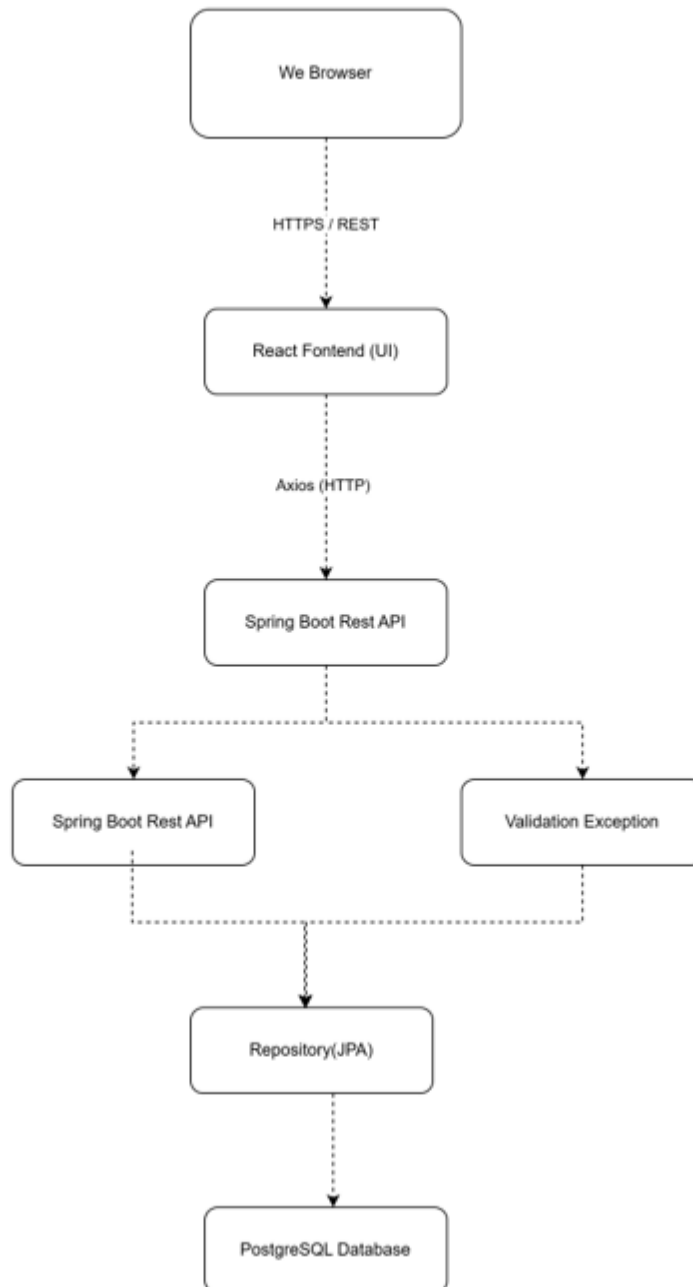
3. High-Level Architecture

3.1 Architecture Overview

The Sales Lead Management Tool follows a **Layered Architecture** that separates the presentation layer, business logic, and data access layer.

This architecture improves maintainability, scalability, and testability while keeping responsibilities clearly separated.

3.2 High-Level Architecture Diagram



3.3 Architecture Description

The application is described from both frontend and backend perspectives, but this repository implements the backend service layer only:

Frontend (context only)

Responsible for:

- Displaying lead information
- Searching leads
- Showing lead details
- Displaying activity timeline
- Adding new activities

Backend (implemented in this repository)

Responsible for:

- Business logic
- Data validation
- Database operations
- REST API
- Error handling
- Logging

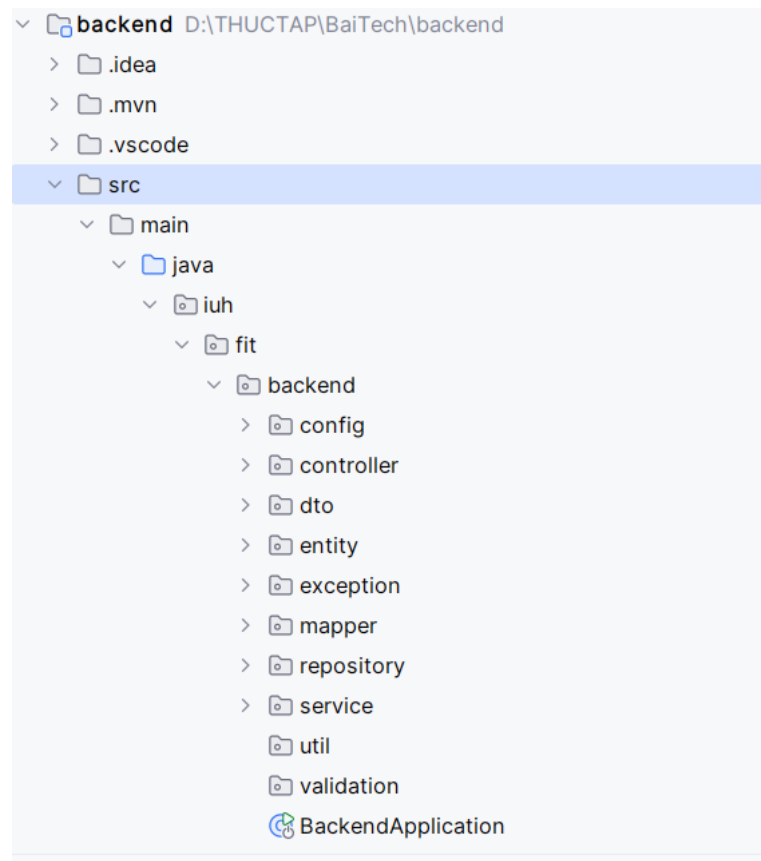
Database

Responsible for:

- Persisting Lead information
- Persisting Activity history
- Maintaining data consistency

4. Component Design

4.1 Backend (implemented in this repository) Components



Controller Layer

Responsibilities

- Receive HTTP Requests
- Validate Request DTO
- Return Response DTO

Service Layer

Contains all business logic.

Responsibilities

- Create Lead
- Search Lead
- Add Activity

- Validate business rules

Repository Layer

Responsibilities

- CRUD operations
- Query Database

Implemented using Spring Data JPA.

Database

PostgreSQL stores

- Leads
- Activities